



VMSCAPE: 在云环境中暴露并利用不完整的分支预测器隔离机制

Jean-Claude Graf[†]
ETH Zurich

Sandro Ruegge[†]
ETH Zurich

Ali Hajiabadi
ETH Zurich

Kaveh Razavi
ETH Zurich

[†]Equal contribution joint first authors

摘要——虚拟化是现代云基础设施的基石，能为客户提供所需的隔离保护。然而，这种隔离正受到推测执行攻击的威胁，CPU厂商试图通过将隔离范围扩展至分支预测器状态来缓解这一风险。我们的系统分析表明，这种扩展不幸并不彻底：虽然现有硬件防护措施已解决了宿主机中客户机控制分支预测这一最明显的案例，但我们发现了AMD Zen 1-5和Intel Coffee Lake处理器上若干新的Spectre分支目标注入（Spectre-BTI）攻击原语，其中某些攻击可使恶意客户机在用户空间执行时控制宿主机的间接分支预测。利用上述原语，我们开发了VMSCAPE——首个Spectre-BTI攻击手段，能使恶意KVM客户机以154 B/s的速度从运行于AMD Zen 5服务器系统的未修改QEMU进程中泄漏任意内存，从而暴露磁盘加密与解密所需的密码密钥。我们对潜在缓解策略的分析表明：在常见场景下，通过选择性刷新分支预测器，可在不影响性能的前提下有效降低VMSCAPE。

1. 引言

Spectre v2或分支目标注入（BTI）[1]攻击近期备受关注，其典型攻击方式是从非特权用户处泄露特权内存[2]。[3]、[4]、[5]、[6]、[7]、[8]尽管这些攻击充分暴露了安全边界被突破的情况，但由于其假设攻击者仅能在用户系统上执行本地代码，因此在实际应用中的影响有限。更具威胁性的场景出现在云端：恶意的虚拟机（VM）可利用Spectre-BTI漏洞泄露主机或其他虚拟机中的信息。然而，目前公开报道的此类攻击案例均要求攻击者在主机上控制代码运行[1]。[5][8]这是一个不切实际的假设。通过对虚拟化边界间分支预测状态隔离的系统性分析，我们发现了多种基于Spectre-BTI的新攻击原语。随后，我们利用其中一种原语构建了首个Spectre-BTI漏洞利用工具，该工具能够从运行默认配置下未修改软件的主机中泄露信息。

云中的Spectre-BTI攻击。以虚拟机形式存在的虚拟化技术，是云环境中安全隔离共置工作负载的主要机制，能够同时保护客户数据和底层基础设施[9]。Spectre攻击（尤其是Spectre-BTI）可通过滥用CPU内部共享的分支预测器状态来破坏这种隔离性[1]。在此场景下常见的威胁模型是恶意虚拟机对管理程序或其他虚拟机发起攻击。因此，硬件和软件供应商特别注重通过以下方式缓解此类威胁：对虚拟机与管理程序中学习到的分支预测条目进行差异化标记，并在切换不同虚拟机时清除分支预测器状态。而针对管理程序的混淆代理攻击所形成的残余攻击面，在不假设代码修改的情况下实际难以被利用[5]。本文研究的核心问题是：是否存在其他尚未被探索的、由恶意虚拟机发起的潜在攻击手段。

适用于云环境的新型 Spectre-BTI 攻击原语。 我们得出一个关键结论：现有的分支预测器隔离机制过于粗粒度——它们将虚拟机监控程序和虚拟机视为单一整体，而实际上它们各自拥有独立的权限级别。为探究现有分支预测器隔离机制如何处理不同虚拟化域中的这些权限级别，我们系统分析了 x86 CPU 中攻击者与受害者防护域的所有可能组合，即 *主机管理程序* (HS)、*主机用户* (HU)、*客户端管理程序* (GS) 和 *客户端用户* (GU)。我们的分析表明：尽管存在硬件层面的防护措施，所有 AMD Zen 处理器（Zen 1 至 Zen 5）和 Intel Coffee Lake 架构均易受新型 *基于虚拟化的 Spectre-BTI* (vBTI) 攻击原语的影响，该攻击发生在客户端用户与主机用户之间，我们将其称为 vBTI_{GU→HU} 和 vBTI_{HU→GU}。vBTI_{GU→HU} 使得恶意虚拟机能够对主机上运行的用户进程发起新的 Spectre-BTI 攻击；而 vBTI_{HU→GU} 则在假设主机具有恶意性且客户机部署了硬件辅助隔离机制（例如 *安全加密虚拟化 - 安全嵌套分页* (SEV-SNP) [10] 或 *信任域扩展* (TDX) [11]）的情况下，同样能引发新的 Spectre-BTI 攻击。此外，我们还发现了其他需要针对本文将讨论的其他攻击场景加以考虑的 vBTI 原语。

vmscape。为展示我们vBTI原语的实际可行性和严重性，我们提出了 VMS CAPE——首个基于Spectre的端到端攻击方案：恶意客户机用户无需任何代码修改且在默认配置下，即可从主机域的虚拟机监控程序中泄露任意敏感信息。我们的攻击目标是广泛使用的内核虚拟机（KVM）/ QEMU [12][13]作为虚拟机监控程序，特别是 QEMU 作为主机中虚拟机监控程序的用户空间组件。虽然vBTI GU→HU原语验证了该攻击方案的可行性，但我们还解决了若干额外挑战，从而能够对AMD Zen 4和Zen 5 CPU实施可靠的端到端攻击。例如，实现任意内存泄漏需要足够大的推测窗口；然而，对于客户机用户而言，获取大型推测窗口极具挑战性——由于无法访问物理内存或对 QEMU 进行代码修改，他们无法直接清除受害分支指针。为解决这一问题，我们对AMD Zen 4和Zen 5的缓存层次结构进行了逆向工程，为其非全包含的最后一级缓存（LLC）构建了驱逐集，从而为 VMS CAPE提供了足够大的推测窗口，使其能够成功应用我们在 QEMU 中发现的技术手段。

VMS CAPE在AMD Zen 5处理器上可以每秒154字节/的速度泄露 QEMU 进程的内存。我们使用VMS CAPE在102秒内定位并泄露了机密数据，例如提取用于磁盘加密/解密的密码学密钥。分析表明，缓解 VMS CAPE需要在特定条件下显式清空分支预测单元（BPU）的状态；具体而言，在进入用户空间虚拟化层之前，每个 VM-EXIT 都必须设置间接分支预测屏障（IBPB）。评估结果显示，Linux内核维护者为应对 VMS CAPE而开发的此类缓解措施，在常见场景下仅会产生轻微的性能开销。

贡献。 我们做出以下贡献：

- 这是首次对所有可能的虚拟化与权限边界下的分支预测器隔离现象进行系统性分析，从而发现了即使在近期CPU防护措施下仍能有效运作的新基本单元。
- 首个针对未修改软件的从客户端到主机的Spectre- BTI攻击——名为 VMS CAPE——利用我们新发现的vBTI GU→HU原语，以及关于AMD Zen 4和Zen 5处理器缓存驱逐机制的新见解，从而扩大了推测窗口。
- 针对多种微架构的缓解方案进行分析，最终提出 IBPB -on- VMEXIT 作为减轻 VMS CAPE影响的基础方法。

责任性披露。 我们已于2025年6月7日向AMD和Intel PSIRT 披露了 VMS CAPE漏洞，该漏洞的披露信息在2025年9月11日前仍处于保密状态。Linux维护团队根据我们的 IBPB -on- VMEXIT 建议，通过补丁修复了 VMS CAPE漏洞。我们已验证这些补丁能有效阻止vBTI原语的攻击行为。VMS CAPE漏洞的追踪编号为 CVE -2025-40300。更多相关信息（包括实验及漏洞利用代码的源码）可访问<https://comsec.ethz.ch/vmscape>获取。

2. 背景

我们介绍了Linux中基于内核虚拟机（KVM）的虚拟化堆栈背景（第2.1节）、分支预测机制（第2.2节）、Spectre- BTI 攻击（第2.3节）及其相应的缓解措施（第2.4节）。最后，我们强调了在云环境中研究 Spectre- BTI 攻击的必要性（第2.5节）。

2.1. KVM 虚拟化

硬件虚拟化扩展技术，例如AMD SVM [10]和Intel VT-x[14]，能够为在同一处理器上同时运行多个操作系统提供硬件支持，以极低的性能开销实现强大的隔离保障。虚拟机监控程序为每个客户虚拟机营造出完全掌控物理系统的错觉。在Linux系统中，KVM 充当虚拟机监控程序，并作为内核模块运行于主机之上；该程序依赖用户空间组件来管理虚拟机的生命周期及模拟设备，我们将此用户空间进程称为虚拟机监控程序的用户模式组件（Hypervisor user）。QEMU [13]是Linux系统中广泛使用的虚拟机监控程序user[15]，其功能包括创建、启动、停止和迁移虚拟机，并模拟所需的硬件设备。每个客户机虚拟CPU（vCPU）在虚拟机监控程序用户空间中对应一个专用线程，这使得主机内核能够像调度普通主机进程一样来安排客户机进程的执行——因为在Linux中，调度器对线程和进程的处理方式是相似的。因此，客户机线程的执行会与其他客户机线程及主机进程的执行交替进行。

进入与退出虚拟机。 vCPU线程以“客户机模式”或“主机模式”运行。要从主机模式切换到客户机模式，该线程会向 KVM 内核模块发出KVM_RUN ioctl指令，并传递其vCPU的指针。根据平台不同，KVM 会执行vmlaunch/vmresume（在Intel平台上）或VMRUN（在AMD平台上）指令。CPU从vCPU中恢复自身状态并启动客户机执行。当客户机执行敏感或特权指令时，CPU会拦截该指令并触发VMEXIT。CPU将内部状态保存回vCPU结构后，在 KVM 中继续执行。根据 VMEXIT 类型的不同，KVM 要么直接处理退出操作，要么通过KVM_EXIT将其委托给虚拟机监控程序用户。

保护域。 为确保机密性和完整性，计算系统必须强制实施在不同保护域中运行的实体之间的隔离。在许多场景下，重点在于区分用户与监督者域（即环0与环3）。然而，在考虑虚拟化环境时，这一抽象概念必须扩展以涵盖客户机执行。如图1所示，用户域包含主机用户（HU）和客户机用户（GU）域，而监督者域则涵盖主机监督者（HS）和客户机监督者（GS）域。

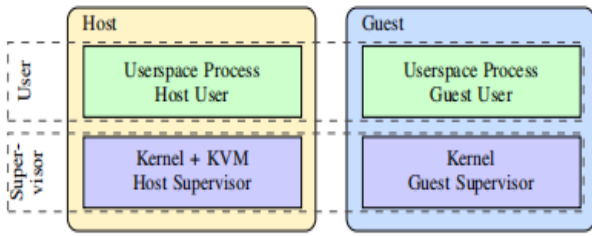


图1. 在配备虚拟机（VM）的x86-64系统中，保护域包括HU、HS、GU和GS，其范围超越了用户模式和管理员模式。

2.2. 分支预测

CPU优化的核心之一在于对分支指令及其目标地址的精准预测。现代CPU中的分支预测单元（BPU）结构极为复杂，由多种针对不同分支类型和上下文设计的预测器组成。虽然静态预测器仅考虑分支的源地址，但动态预测器会利用分支路径的历史信息作为额外上下文参考[2]。3], [6], [7],[16], [17] 间接分支 会将控制流转移至由寄存器或内存指定的目标地址。这使得分支指令能够指向多个目标地址，BPU 会尝试根据分支路径的历史记录进行关联判断。英特尔CPU采用分支目标缓冲区（BTB）进行静态预测，以及间接分支预测器（IBP）进行动态预测。在可用情况下，动态预测优先于静态预测[8]。17]在AMD CPU上，动态预测器被称为间接目标数组（ITA），当分支操作涉及多个目标地址时使用该机制；否则仅使用BTB[18]。路径历史记录会同时存储在Intel和AMD架构的分支历史缓冲区（BHB）中。图2展示了间接分支预测所涉及的结构。

2.3. Spectre-BTI

Spectre Branch Target Injection 或 Spectre- BTI [19] 是一类推测执行攻击，攻击者可通过利用间接分支预测错误，在保护域边界之间泄露数据。攻击者使BPU将目标分支（作为推测装置的一部分）预测为披露装置，从而引发瞬态执行。在披露装置中，秘密信息的访问方式使得攻击者能够通过侧信道（通常基于缓存的FLUSH+RELOAD[20]）事后推断出该秘密。为导致目标分支的预测错误，攻击者需使用一个与目标预测器中的目标分支发生冲突的训练分支来训练BPU。若要针对特定预测器，攻击者还需模拟训练分支中目标分支的路径历史。

此前关于Spectre- BTI的大多数研究都集中在以下场景：非特权主机用户进程针对主机监督程序发起攻击[1]。2], [3], [5], [6], [21]相比之下，用户进程之间的攻击却鲜少受到关注[7]。

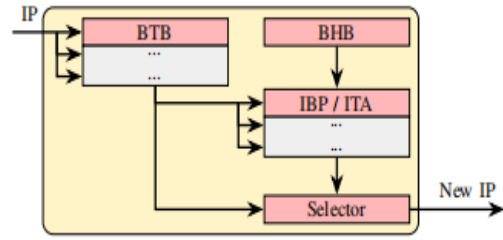


图2. 参与预测间接分支的BPU组件。BTB提供静态预测，而IBP/ITA则将目标与存储在Intel/AMD处理器BHB中的路径历史信息相关联。

2.4. Spectre- BTI 缓解措施

Spectre-分支目标注入（BTI）攻击要求攻击者控制用于预测受害者的BTB状态，这意味着攻击者与受害者域之间需要共享BPU状态。主域隔离技术旨在通过阻止关键保护域边界之间的BPU状态共享来缓解BTI攻击。间接分支限制推测（IBRS）[22]可防止低权限域（例如用户）的分支影响高权限域（例如管理员）的分支，从而保护内核免受用户模式攻击者的侵害。英特尔和AMD已部署增强型IBRS（eIBRS）[22]和自动IBRS（AutoIBRS）[23]以改进IBRS；这两种防护措施均为持续激活，且无需在权限转换时进行昂贵的模型特定寄存器写入操作。这些对BPU进行清洗的防护措施可在潜在恶意条目影响其他保护域中的受害者之前将其清除。间接分支预测屏障（IBPB）允许开发人员在那些原本无法充分隔离的域转换期间显式清除BPU状态。在启用了Si多线程支持（SMT）的CPU上，BPU状态也可能在同级线程之间共享。单线程间接分支预测器（STIBP）[24]是一种缓解措施，可限制SMT线程之间的BPU状态共享。

不同微架构提供的缓解措施各不相同，默认的缓解方案取决于硬件支持和内核配置。表1列出了我们在评估的系统中可用的缓解措施，并标明了这些措施在Ubuntu 24.04系统上是否默认启用。

2.5. 动机

虽然用户对内核发起的Spectre- BTI攻击属于安全漏洞，但此类攻击需要在受害机器上执行代码，从而限制了其影响范围。可以说，Spectre- BTI更具威胁性的应用场景在于云端环境——攻击者可轻松租用虚拟机，并针对虚拟机监控程序或其他虚拟机实施攻击。相关前期研究[1]5]研究人员已探讨了恶意攻击者针对虚拟机监控程序发起攻击的情形。然而，此类攻击需通过修改虚拟机监控程序的代码来植入恶意组件，这极大限制了其实际应用价值与攻击效果。由此引出一个关键问题：是否能在不做出过于严苛假设的前提下，跨虚拟化边界实施Spectre- BTI攻击？

TABLE 1. EVALUATED MICROARCHITECTURES AND THEIR ISOLATION MECHANISMS AGAINST SPECTRE-BTI.

Vendor	CPU	Year	Codename	Microarchitecture	Microcode	Isolation Mechanism			
						IBPB	IBRS	eIBRS/AutoIBRS	STIBP
Intel	Core i7-8700	2017	Coffee Lake S	Coffee Lake	0xfa	●	●	○	●
	Core i7-13700K	2022	Raptor Lake	Raptor Cove	0x12c	●	○	●	●
				Gracemont	0x12c	●	○	●	●
AMD	Ryzen 5 1600X	2017	Summit Ridge	Zen 1	0x8001137	●	●	○	○
	EPYC 7252	2019	Rome	Zen 2	0x8301038	●	●	○	●
	EPYC 7413	2021	Milan	Zen 3	0xa0011d3	●	●	○	●
	Ryzen 7 7700X	2022	Raphael	Zen 4	0xa601209	●	●	●	●
	Ryzen 5 9600X	2024	Granite Ridge	Zen 5	0xb404023	●	●	●	●

○ Not available ● Available ● Available and Default (Ubuntu 24.04)

3. 威胁模型

攻击者的目标是推断跨越主机-客户机虚拟化边界的信
息。具体而言，我们考虑以下三种威胁模型：首先， $TM_{G \rightarrow H}$ 描述了控制客户机虚拟机的攻击者针对主机系统的场
景；其次， $TM_{H \rightarrow G}$ 表示恶意主机针对系统上运行的客户机
发起攻击——这种威胁模型在涉及依赖英特尔信任域扩展
(TDX)或AMD安全加密虚拟化-安全嵌套分页 (SEV -
SNP)来抵御恶意主机的客户机时尤为相关；第三，恶意
客户机并非直接攻击主机，而是可能攻击另一个客户机，
这体现在 $TM_{G1 \rightarrow G2}$ 模型中。我们假设采用了硬件虚拟化扩展
技术：AMD使用 SVM [10]，Intel使用 VT-x [14]。此外，我
们假设所有组件（特别是主机和客户机内核以及虚拟机监
控程序_{用户端}）均不存在软件漏洞。最后，我们假设这些系
统采用了针对Spectre- BTI的所有默认安全防护措施，具体
如表1所示。

4. 挑战概述

现有的Spectre- BTI缓解措施旨在跨保护域边界隔离
BPU状态。尽管过去已研究过恶意客户机攻击虚拟机监控
程序这一最典型的案例[1]，[5]目前仍缺乏针对虚拟化环境
中BPU状态隔离的系统性分析，该分析需综合考虑客户机
与主机的不同权限级别。这引出了我们的首要挑战：

挑战C1： 识别 BPU 在虚拟化环境中保护域隔离机制存在的漏洞。

为应对这一挑战，我们需要仔细评估 BPU 中各个结
构的隔离效果。在第5节中，我们系统性地测试了 BTB
和 IBP/ITA在考虑客户机与主机不同权限级别时所提供
的隔离能力。系统分析表明，许多近期提出的微架构易
受某些基于虚拟化的Spectre- BTI (vBTI)原语攻击影
响，这说明 BPU 缺乏有效的主机-客户机隔离机制。

然而，利用这些新型vBTI基本单元所带来的具体后
果及潜在攻击风险尚不明确。我们的下一个挑战是确定
在何种特定情境下，攻击者能够在不同的威胁模型中实
施有效的攻击。

挑战C2： 在虚拟化环境中，理解并识别受多孔 BTB
隔离机制影响的不同威胁模型。

应对这一挑战需要对攻击者利用已识别的vBTI原语
所需条件进行详细分析。在第6节中，我们基于第3节提
出的三种威胁模型，提出了五种新型攻击场景。这些攻
击场景使攻击者能够利用所识别的原语跨越虚拟化边界
对受害者发起攻击。其中，我们重点研究了一种新型跨
权限vBTI原语的攻击方式：恶意客户机攻击虚拟机监控
程序_{用户端}以泄露敏感信息（例如云基础设施密钥）。尽
管底层推测原语在概念上较为简单，但构建实用的端到
端攻击方案仍面临挑战。

挑战C3： 实现跨虚拟化边界的实际应用与数据泄露。

客户机与主机软件层之间的交互有限，这使得寻找
合适的推测工具和可靠的侧信道变得极具挑战性。实现
有效数据泄露的关键在于确保足够的推测窗口宽度。然
而，在现代AMD Zen微架构（尤其是Zen 4和Zen 5）上
实现有效的缓存驱逐仍是一个未解决的问题。第7节详
细阐述了在AMD Zen 4和Zen 5平台上构建首个可靠缓存
驱逐机制的方法。基于这些研究成果，第8节开发了
VMS CAPE——一种实用的Spectre漏洞利用方案，它允
许客户机通过我们新发现的vBTI原语之一，在虚拟化边
界之外泄露主机内存。

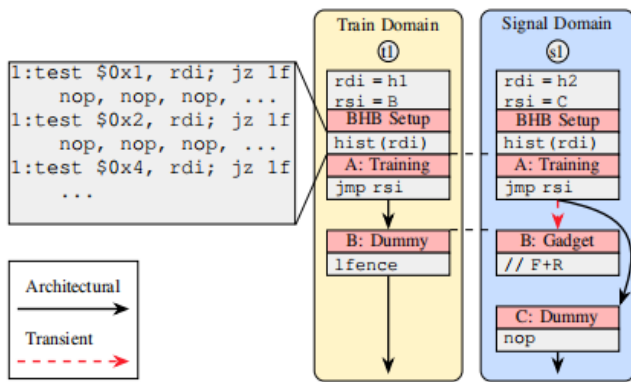


图3. vBTI实验中的训练 (t1) 与信号传输 (s1) 轨迹使我们能够评估 BPU 在训练域与信号传输域边界之间的状态隔离能力。间接分支的源节点A和目标节点B在两个域中映射至相同的虚拟地址。每个间接分支前均会执行 BHB 设置——即通过rdi中的种子值生成一系列分支操作，使两条轨迹中的 BHB 状态保持一致。这确保了 BPU 无法区分训练分支与信号传输分支，因为两者会在 BTB 及 ITA/ IBP 阶段发生冲突。若隔离机制失效，信号传输分支将偏离由rsi定义的架构目标，并在B处临时执行已训练的目标指令。我们通过一种信息披露机制来检测这种误判现象。

5. 虚拟化环境下的 BPU 隔离

我们分析了在虚拟化环境中，针对多种微架构跨越保护域边界进行间接分支预测的隔离性能。为评估整体隔离保障能力，我们需要分别考察静态与动态间接分支预测器的隔离效果，并测试x86架构硬件虚拟化环境下四个基本保护域（即HU、HS、GU和GS）之间的相互干扰。所有评估系统详见表1。近期英特尔Raptor Lake等处理器采用异构设计，包含两种微架构：一种专为高性能任务优化，另一种则适用于低功耗场景。

5.1. 方法论

为评估间接分支的跨域 BPU 干扰，我们采用经典的 Spectre- BTI [1] 实验方法（如图3所示）。我们在一个域进行训练，并在另一个域使用 F LUSH+R ELOAD 侧信道 [20] 生成干扰信号。

本次分析要求我们在所有四个评估域中执行任意代码。对于主机系统（HS）的代码执行，我们实现了一个内核模块，允许用户以超级管理员身份运行任意用户代码；这需要在主机系统上禁用超级管理员模式执行防护（SMEP）和超级管理员模式访问防护（SMAP）。对于客户机系统（guest）的执行，则依赖于Linux内核内置的KVM自测试基础设施。这一简化的虚拟机监控程序使我们能够将代码映射到任意客户机虚拟地址并执行。然而，KVM自测试默认以GS权限运行，因此我们需要扩展该框架，并实现额外的权限转换机制以达到GU权限。

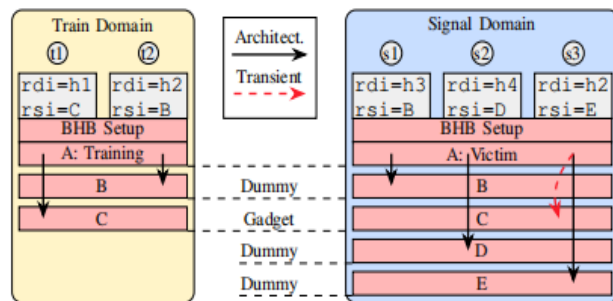


图4. 通过组合不同的训练轨迹 (t1、t2、s1、s2) 我们可以诱导 BPU 使用来自 BTB 或 IBP /ITA 的信号轨迹s3的预测结果。分支源A与目标B和C的虚拟地址在两个域中完全一致。

在我们的评估过程中，至关重要的是要验证所有参与间接分支预测的结构是否处于隔离状态。这些结构包括英特尔架构中的 BTB 和 IBP，以及AMD架构中的 BTB 和 ITA。逆向工程要求我们控制 BHB 的状态，以引发或防止 IBP /ITA 中的冲突。如图3所示，我们通过执行基于控制种子生成的伪随机分支序列来初始化 BHB。这段历史记录设置代码直接置于训练分支和受害分支之前，从而能够精确控制这些分支处的 BHB 状态。

请注意，本节所述的所有修改均仅用于逆向工程目的。第8节中介绍的最终攻击无需任何软件修改，并可在表1所列的默认防护措施下执行。

BTB 预测。 如图2所示，多个结构参与间接分支的预测。因此，需要精确的实验设置以确保预测结果来自 BTB 而非 IBP /ITA。英特尔始终会在 BTB 中检查命中情况，但在可用时优先考虑 IBP 的命中结果 [17]；而 AMD 则使用 BTB 来预测所有仅接触过单一目标的间接分支，否则会优先采用 ITA 的预测结果 [18]。

在训练域中，从A到B的训练分支执行时，会在Intel和AMD架构上均向 BTB 插入一条记录（图3 t1）。切换至信令域后，我们在A到C处执行信令分支（s1）。由于A在训练域与信令域中是相同的虚拟地址，BTB无法区分两者。此外，使用种子值h₁可实现不同的 BHB 状态。≠ h₂ 确保英特尔处理器上的 IBP 无法提供预测结果，而自动切换至 BTB 进行处理；而在AMD处理器上，由于分支源A迄今仅遇到目标C，因此ITA中尚未创建任何条目，导致该分支同样由 BTB 进行预测。为确保状态清晰，我们在每次迭代之间插入一个 IBPB。本实验中检测到的信号表明该机制存在vBTI攻击风险。

IBP /ITA 预测。 测试动态预测器的关键挑战在于：必须确保预测结果并非源自 BTB。为此，我们需要执行特定的验证流程。

TABLE 2. SUMMARY OF THE OBSERVED vBTI PRIMITIVES FOR ALL EVALUATED SYSTEMS. EMPTY HALVE CIRCLES INDICATE VULNERABLE, WHILE COLORED ONES INDICATE THE DEFAULT MITIGATION PREVENTING A LEAK.

Microarch.	vBTI / vBTI-SMT Primitive									
	HU→GU	HU→GS	HS→GU	HS→GS	GU→HU	GU→HS	GS→HU	GS→HS	GU→GSU	GS→GSU
Zen 1	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Zen 2	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Zen 3	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Zen 4	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Zen 5	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Coffee Lake	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Raptor Cove	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
Gracemont	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤	⬤
same-thread is: ⬤ vulnerable, or protected by: ⬤ retpoline, ⬤ IBPB, ⬤ AutoIBRS/eIBRS										
cross-SMT is: ⬤ vulnerable, or protected by: ⬤ retpoline, ⬤ STIBP										

训练轨迹如图4所示。在训练域中，我们在A处执行训练分支两次，每次针对不同的目标。为迫使BPU区分这两次调用，我们设置了不同的历史种子 $h_1 \neq h_2$ 。因此，不仅会在BTB中创建条目，还会在IBP/ITA内部创建条目。

从训练域切换到信号域可能会导致BTB条目失效或被隔离。然而，在Intel和AMD架构中，都需要BTB中的有效条目才能让IBP/ITA进行预测。因此，我们必须确保在切换至信号域后仍存在相应的BTB条目。在Intel架构中，仅需在A(s1)处遇到一次间接分支即可使后续分支(s3)由IBP进行预测；而在AMD架构中，则需要先出现指向不同目标的额外分支(s2)，才能由ITA对最终分支(s3)进行预测。如果本实验中在当前域转换处检测到信号，但在先前域未检测到信号，则表明BTB与IBP/ITA之间存在不平衡的隔离现象。

5.2. 原始评估

为评估隔离保证性，我们系统性地测试了虚拟化与权限边界之间所有保护域的组合。我们启用了表1中列出的所有默认缓解机制。此外，我们将所有实验分别以间接跳转和间接调用作为训练指令与信号指令各重复一次，从而评估两者是否被相似预测且均能实现有效隔离。结果表明间接跳转与间接调用之间无差异，说明二者均受到同等程度的缓解措施处理；因此在本文后续内容中我们不再区分二者。同时观察到干扰信号在客户机与主机之间呈对称分布，表明隔离机制在双向传输中具有等效性。我们采用符号 $vBTI_{X \rightarrow Y}$ 表示一种训练域为X、信号域为Y的vBTI原语。

表2展示了各微架构对vBTI原语的敏感性。由于BTB与IBP/ITA在漏洞易感性方面未发现差异，详见表2。

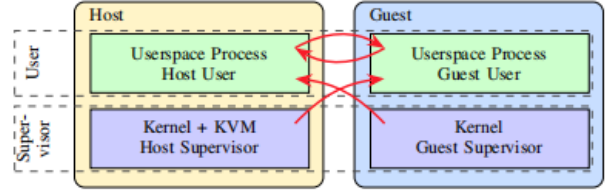


Figure 5. Effective protection domain isolation on Zen 4, susceptible to $vBTI_{GU \rightarrow HU}$, $vBTI_{GS \rightarrow HU}$, $vBTI_{HU \rightarrow GU}$, and $vBTI_{HS \rightarrow GU}$.

无需区分它们。我们观察到，除Intel Raptor Cove和Gracemont外，所有微架构都易受某些vBTI原语攻击。最值得注意的是，Zen 1至Zen 5都易受 $vBTI_{HU \rightarrow GU}$ 和 $vBTI_{GU \rightarrow HU}$ 攻击；而Zen 1至Zen 4还额外易受 $vBTI_{HS \rightarrow GU}$ 和 $vBTI_{GS \rightarrow HU}$ 攻击（如图5所示）。虽然Raptor Cove和Gracemont已得到充分保护，但Coffee Lake对于Zen 1至Zen 4的上述原语同样存在漏洞。然而，我们发现没有任何微架构会受到针对HS和GS域的任何原语攻击。

观察结果 O1. 尽管已启用最新的缓解措施，所有AMD Zen微架构和Intel Coffee Lake都易受 $vBTI_{GU \rightarrow HU}$ 与 $vBTI_{HU \rightarrow GU}$ 的影响。此外，Zen 1至Zen 4以及Coffee Lake均受到 $vBTI_{GS \rightarrow HU}$ 和 $vBTI_{HS \rightarrow GU}$ 的影响。

BPU 隔离机制。 AMD Zen 4和Zen 5采用AutoIBRS作为Spectre-BTI缓解措施。在监督模式下，该机制会阻止对间接分支目标的推测执行，直至这些目标得到解析[5]。[6]通过在禁用AutoIBRS的情况下重新运行之前的实验，我们可以更深入地了解Zen 4和Zen 5微架构上的隔离机制实施情况。在缺乏AutoIBRS的情况下，Zen 4会受到所有vBTI原语的影响；而Zen 5则不受跨越用户与监督者边界原语的影响。这表明AMD Zen 5 CPU在AutoIBRS基础上还具备一种额外的原生隔离机制，其功能相当于为预测条目设置单比特权限级别别标签。

观察结果 O2. 在Zen 5架构中，分支预测条目均标注了单比特权限级别，以区分用户域和监督者域。

然而，单个比特位不足以区分虚拟化环境中存在的四个域。因此，仍需使用AutoIBRS来分离特定域（例如HS域和GS域）。

观察结果 O3. 尽管BTB中增加了隔离位，Zen 5上的BPU仍无法区分主机域与客户机域，因此仍依赖AutoIBRS来保护监督域。

SMT 隔离。为补充先前测试同线程隔离的实验，我们采用改进后的实验来验证vBTI- SMT 原语。我们的训练过程会迭代执行训练分支（图3 t1）；随后信号处理过程绑定 $\textcircled{SMT}$ 兄弟核心，并使用不同历史数据执行信号分支（t2）。信号处理完成后，我们通过 IBP $\textcircled{B}$ 刷新 BPU，使其忽略设备目标信息。如表2所示：虽然Zen 2至Zen 5及Raptor Lake架构上的 STIBP 能成功隔离兄弟线程，但Zen 1架构缺乏 STIBP 功能，导致vBTI- SMT_{HU→GU}、vBTI- SMT_{GU→HU}、vBTI- SMT_{HS→GU}及vBTI- SMT_{GS→HU}等原语无法实现隔离。此外我们发现，尽管Coffee Lake支持 STIBP，但默认并非始终启用该功能，从而允许相同的跨 SMT 原语被调用。

观察结果 O4。在 Zen 1 上 STIBP 完全缺失，而在 Coffee Lake 上仅条件性启用；这两种微架构均对 vBTISMT_{GU→HU}、vBTI- SMT_{HU→GU}、vBTI- SMT_{GS→HU} 及 vBTI- SMT_{HS→GU} 存在漏洞。

6. 不同威胁模型下的剥削行为

我们的实验表明，某些虚拟化边界未能得到充分隔离，导致分支目标在这些边界之间发生干扰。在本节中，我们将研究结果与第3节提出的三种威胁模型相结合，并讨论了五种现实且此前未被探索的场景——这些场景中上述发现会导致具体的安全漏洞。我们用 $A \rightarrow B | V$ 表示每种攻击场景，其中 A 和 B 分别代表攻击者和受害者的防护域，而 V 指代具体目标（例如主机进程）。

6.1. 从访客到主机的攻击

我们分析的第一个威胁模型是 $TM_{G \rightarrow H}$ ，该模型描述了恶意访客针对主机系统的攻击行为。在此模型中，我们识别出三种具体场景。

$\textcircled{S1} G \rightarrow H | \text{主机进程}$ 。在此场景中，恶意来宾进程会针对主机用户进程泄露机密信息（例如 SUID 进程的密码哈希值）[7]。受害进程可能在来宾进程被抢占后被调度到同一硬件线程上（即**同一线程**），也可能被调度到同一物理核心上的 SMT 兄弟线程上（即**跨 SMT 线程**）。在同一线程情况下，除了缺乏来宾到主机的 BTB 隔离外，该攻击还要求任务切换期间不存在 BTB 清理机制。根据我们的逆向工程观察，我们发现Zen 1至Zen 5以及Intel Coffee Lake微架构缺乏足够的隔离性，易受vBTI_{GS→HU}和vBTI_{GU→HU}原语的影响。对于跨 SMT 攻击，来宾到主机的 BTB 干扰必须与兄弟线程间的共享预测器状态相结合，这由vBTI- SMT_{GS→HU}和vBTISMT_{GU→HU}原语所体现。这一条件在Zen 1和Coffee Lake微架构上均得到满足。

尽管这些攻击在理论上是可行的，但实际实施时面临诸多挑战。基于客户机的攻击者通常对主机用户进程的控制力有限，因此难以可靠地操控或与其交互。在客户机与任意主机用户进程之间建立可靠的侧信道尤为困难，因为攻击者既缺乏空间控制也缺乏时间控制。具体而言，攻击者无法决定目标进程何时何地调度执行，这进一步增加了精确利用攻击漏洞的难度。

$\textcircled{S2} G \rightarrow H | \text{Hypervisor}_{\text{user}}$ 。该场景考虑了一个恶意客户机针对其自身的 Hypervisor_{user}。这要求 VMEXITs 和 KVM_EXITs 的 BTB 清理不足，同时在同一线程场景下客户机与主机之间缺乏 BTB 状态隔离。与一般的 $G \rightarrow H | \text{Host Process}$ 攻击相比，此处的攻击方式更为简单：客户机可故意触发由其 Hypervisor_{user} 处理的 VMEXITs，从而在相同 SMT 场景下实现频繁的攻击者-受害者交互；跨 SMT 场景也更具实际可行性——通常调度机制通过降低攻击者与受害者共处的概率来缓解 crossSMT BTI 攻击，但在此情况下，受害者会执行同一客户机的 Hypervisor_{user}，使得共处概率显著增加。尽管攻击者仅能访问映射到虚拟机监控程序_{用户}地址空间的内存，但其中仍可能包含敏感数据，例如用于网络或存储的云基础设施密钥[25]。我们发现Zen 1至Zen 5及Coffee Lake架构均存在此类漏洞，其中Zen 1和Coffee Lake还存在跨 SMT 攻击风险。

观察结果 O5：Cross- SMT $G \rightarrow H | \text{Hypervisor}_{\text{用户}}$ 攻击更容易发生，因为攻击者与受害者属于同一客户机执行环境，这使得共置攻击的可能性显著增加。

$\textcircled{S3} G \rightarrow H | \text{客户端内核}$ 。在此场景中，攻击者利用虚拟机监控程序_{用户}作为受误导的代理，从自身客户实例中提取内核内存。因此，该场景假设攻击者是运行在客户机上的无特权进程。这是 $G \rightarrow H | \text{虚拟机监控程序}_{\text{用户}}$ 场景的一种变体，但其泄露对象并非虚拟机监控程序_{用户}的密钥，而是映射在虚拟机监控程序_{用户}中的客户机自身内存。除要求虚拟机监控程序_{用户}对客户机内存进行映射外，其余前提条件与 $G \rightarrow H | \text{虚拟机监控程序}_{\text{用户}}$ 场景完全相同。具体而言，这些前提条件包括缺乏客户机与主机之间的 BTB 隔离机制，以及对 VMEXIT 和 KVM_EXIT 事件的清理处理不充分。

观察结果 O6：近期推出的多种微架构（包括 Zen 5）均易受 $G \rightarrow H | \text{Guest Kernel}$ 攻击的影响。

TABLE 3. ATTACK SCENARIOS BY THREAT MODELS, INDICATING REQUIREMENTS, TARGET, AND VULNERABLE MICROARCHITECTURES.

Scenario	Target	SMT	Required Primitives	Exploitable on							
				Zen 1	Zen 2	Zen 3	Zen 4	Zen 5	Coffee Lake	Raptor Cove	Gracemont
① G→H Host Process	Host user process	Same	vBTI _{GS→HU} or vBTI _{GU→HU}	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	vBTI-SMT _{GS→HU} or vBTI-SMT _{GU→HU}	✓	✗	✗	✗	✗	✓	✗	✗
② G→H Hypervisor _{user}	Hypervisor _{user}	Same	vBTI _{GS→HU} or vBTI _{GU→HU}	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	vBTI-SMT _{GS→HU} or vBTI-SMT _{GU→HU}	✓	✗	✗	✗	✗	✓	✗	✗
③ G→H Guest Kernel	Guest supervisor	Same	vBTI _{GU→HU}	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	vBTI-SMT _{GU→HU}	✓	✗	✗	✗	✗	✓	✗	✗
④ H→G SEV-SNP Process	Guest user process	Same	vBTI _{HU→GU} or vBTI _{HS→GU}	-	-	✓	?	✗	-	✗	✗
		Cross	vBTI-SMT _{HU→GU} or vBTI-SMT _{HS→GU}	-	-	✗	✗	✗	-	✗	✗
⑤ G ₁ →G ₂ Guest Process	Other guest user process	Same	vBTI _{G₁U→G₂U}	✗	✗	✗	✗	✗	✗	✗	✗
		Cross	vBTI-SMT _{G₁U→G₂U}	✓	✗	✗	✗	✗	✓	✗	✗

✓ Vulnerable ✗ Not vulnerable - Not applicable ? Not supported on the tested system

6.2. 从主机到客户端的攻击

我们分析的第二个威胁模型是 $TM_{H \rightarrow G}$ ，该模型中恶意主机会针对客户端发起攻击。

④ $H \rightarrow G$ | SEV-SNP 过程。在此场景中，恶意主机会针对客户机发起攻击——该客户机通常伪装成敌对主机，并采用基于硬件的隔离技术（如 SEV-SNP [10] 或 TDX [11]）。与 $TM_{G \rightarrow H}$ 不同，攻击者能够控制客户线程的调度，从而对受害者拥有更多控制权。由于这些技术会引入额外的安全措施，在未部署 SEV-SNP/TDX 的系统上，仅凭工作中的 vBTI_{HS→GU}、vBTI_{HU→GU}、vBTI-SMT_{HS→GU} 及 vBTISMT_{HU→GU} 原语不足以判定不同微架构的漏洞存在性。然而我们已确认：在 Zen 3 架构上，默认情况下 SEV-SNP 并未实现客户机与主机之间的 BTB 隔离；而在 Zen 5 架构中，我们观察到预测结果在主机与 SEV-SNP 客户机之间实现了有效隔离。遗憾的是，我们目前缺乏 Zen 4 服务器来验证这一隔离效果。虽然 Zen 1 和 Zen 2 不支持 SEV-SNP，但在使用 SEV 或 SEV-ES 等较弱技术的场景中，它们可能存在安全漏洞。

观察结果 O7： AMD Zen 3 存在 $H \rightarrow G$ | SEV-SNP 攻击漏洞。

SEV-SNP 为访客提供了多种可选保护机制，我们将在第 9.3 节中详细讨论。我们的分析表明，早期的、eIBRS 之前的 Intel 微架构对 vBTI_{HS→GU}、vBTI_{HU→GU}、vBTI-SMT_{HS→GU} 以及 vBTISMT_{HU→GU} 这些基本操作存在漏洞；但这些系统并不支持 TDX。

6.3. 宾客之间的攻击

我们分析的第三个威胁模型是 $TM_{G_1 \rightarrow G_2}$ 。在此模型中，恶意客户端会攻击在同一主机系统上运行的另一个客户端。

⑤ $G_1 \rightarrow G_2$ | 客户进程。在此场景中，恶意客户进程会针对另一个客户进程的用户进程发起攻击，这凸显了不可信虚拟机之间共存的风险。这表明不同客户进程之间缺乏 BTB 隔离。与先前场景类似，我们区分了同线程和跨 SMT 两种情况：在同线程情况下，受害者会在攻击者线程被抢占后，于同一线程上执行；此外还要求虚拟机切换点缺乏 BTB 清理机制——不过 Linux 内核通过虚拟机切换点上的 IBPB 功能对此进行了缓解；而跨 SMT 情况则需要 BTB 状态在兄弟线程间共享。根据我们在第 5.2 节中的逆向工程分析，Zen 1 和 Coffee Lake 架构均缺乏 SMT 线程隔离机制，因此易受 $G_1 \rightarrow G_2$ | 客户进程攻击的影响。

观察结果 O8。 AMD Zen 1 和 Intel Coffee Lake 存在易受 $G_1 \rightarrow G_2$ | Guest Process 攻击的漏洞。

然而，实际应用仍面临诸多挑战。在公共云主机上，攻击者与客户虚拟机之间通常缺乏直接的交互通道，也无法利用共享内存实现侧信道攻击。

6.4. 摘要

表 3 概述了各类攻击场景，总结了其前提条件、目标对象及易受攻击的微架构。我们的分析表明，vBTI 原语可在第 3 节介绍的三种威胁模型中均导致具体的安全漏洞。这些攻击包括：恶意客户机对主机发起的攻击（ $TM_{G \rightarrow H}$ ）、恶意主机对客户机发起的攻击（ $TM_{H \rightarrow G}$ ），甚至两个客户机之间的攻击（ $TM_{G_1 \rightarrow G_2}$ ）。

为证明 vBTI 攻击的可行性，我们推出了 VMS CAPE——一种针对 $G \rightarrow H$ | Hypervisor_{user} 场景的端到端漏洞利用工具。该场景代表了

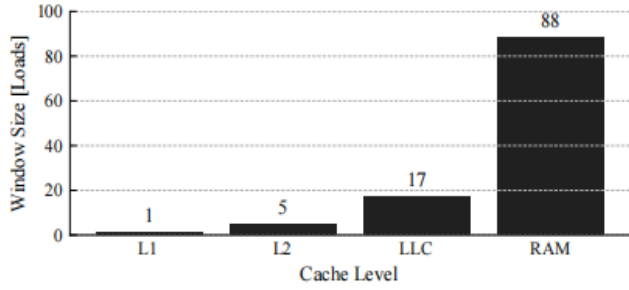


Figure 6. Speculation window size on Zen 4 as the maximum number of executed dependent loads with the pointer to the indirect branch destination residing in the respective cache level.

这对云服务提供商而言尤为严重，因为虚拟机监控程序用户内存的泄露直接针对其基础设施。

7. 扩大投机窗口

我们的目标是在 $G \rightarrow H$ Hypervisor_{user} 场景下构建一个端到端的内存泄漏漏洞利用方案，针对最新的 AMD Zen 4 和 Zen 5 系统。在构建该漏洞利用方案的过程中，我们发现受害分支的推测窗口过短。首先，我们将阐述如何评估推测窗口的大小；随后，讨论如何通过缓存驱逐机制来扩大该窗口。

预测窗口大小。 为测量预测窗口的大小，我们设计了以下实验：使用一个从内存获取目标地址的间接分支指令，从而使 BPU 能够对目标地址进行预测和推测。预测窗口的大小取决于目标地址在架构层面的解析速度，而该速度又取决于存储间接分支目标地址的缓存层级。我们可以通过在预测过程中连续执行 n 次依赖性加载操作，并检查最后一次加载是否命中缓存来测量预测窗口的大小；同时确保除最后一次加载外的所有加载操作均位于 L1 缓存中。对于每个 $n \in [1, 127]$ ，我们重复实验 4096 次，并调整存储间接分支目标地址的缓存层级。

如图 6 所示，当目标地址位于 L1 层级时，在推测窗口期间仅执行一次依赖性加载操作。对于基于 F LUSH+R ELOAD 的侧信道攻击而言，这种推测窗口长度是不足的，因为该攻击至少需要两次依赖性加载操作。虽然我们在 L2 层级的研究结果表明存在足够大的推测窗口以实现完美的推测装置，但某些攻击仍需要更长的推测窗口。这可能是由于先前指令产生的额外竞争延缓了披露装置的运行速度，或是因为受害软件在指令流中更早阶段就开始加载目标地址（例如用于验证其是否为有效指针）。这些因素会缩短披露装置可用的推测时间，迫使攻击者扩大整体推测窗口的范围。

可通过清除存储间接分支目标地址的内存来延长推测窗口。

TABLE 4. CACHE DETAILS ON AMD ZEN 3, ZEN 4 AND ZEN 5. THE SIZES MAY VARY BETWEEN PROCESSORS OF THE SAME GENERATION.

Core	L1			L2			Last Level Cache (LLC)		
	Size	Sets	Ways	Size	Sets	Ways	Size	Sets	Ways
Zen 3	32KiB	64	8	512KiB	2048	8	32MiB	32768	16
Zen 4	32KiB	64	8	1MiB	2048	8	32MiB	32768	16
Zen 5	48KiB	64	12	1MiB	1024	16	32MiB	32768	16

从缓存中读取数据会导致受害分支的解析速度变慢[1]。然而，虚拟机内的攻击者无法直接清除目标指针，因为支持该指针的物理内存是分配给主机进程的，虚拟机无法访问。因此，攻击者必须将目标指针从缓存层次结构中驱逐出去。需要强调的是，驱逐操作应仅针对必要的缓存集，以避免在推测阶段拖慢整个受害程序的执行速度。

7.1. 在 AMD Zen 4 和 Zen 5 上的 LLC 强制清除

Wang 等人最近的研究[26] 我们验证了在 AMD Zen 3 架构上构建 LLC 置换集的可行性。基于他们的研究成果，我们首次为 AMD Zen 4 和 Zen 5 架构实现了 LLC 置换机制。为实现这一目标，我们遍历了缓存层次结构的所有层级，识别出与 Zen 3 架构的差异，并逐步学会置换更大层级的缓存单元，从而延长推测窗口时间。随后，我们展示了如何将该置换集构建流程移植至虚拟机环境，并利用 XOR 索引技术显著提升流程效率与可靠性。表 4 列出了所评估处理器的相关缓存架构参数。为简洁起见，本文首先详细阐述 Zen 4 架构，再总结其与 Zen 5 架构的差异。

在逆向工程中，我们采用 1GB 分页机制以确保虚拟地址与物理地址之间的位匹配。此外，我们还利用了 AMD 精确的 APERF 计时器，该计时器可通过 rdpru 指令访问[27]。默认情况下，1GB 分页机制及精确计数器均不对客户机开放。因此，我们仅在逆向工程过程中使用这些功能，而不适用于第 8 节所述的端到端漏洞利用方案。

缓存访问延迟。 我们首先对每个缓存层级的内存访问时序函数进行校准，从而确定内存访问是由 L1、L2、LLC 还是主内存提供的。为计算延迟，我们按以下步骤进行：从包含 N 个缓存行的内存区域开始，以伪随机顺序访问所有缓存行并记录访问时间；随后逐步增大 N 值重复此过程。图 7 展示了随着内存区域大小 N 增大而变化的中位访问时间曲线，其中平台期对应的延迟分别表示 L1、L2、LLC 和 RAM 的访问延迟。

L1 驱逐机制。 根据我们先前的缓存访问测量结果，可以确认 Zen 4 的 L1 索引方案与 Zen 3 相同。通过访问 8 个不同的缓存行（对应 L1 的各路），即可驱逐任何 L1 条目；这些缓存行均共享相同的位 [6:11]。

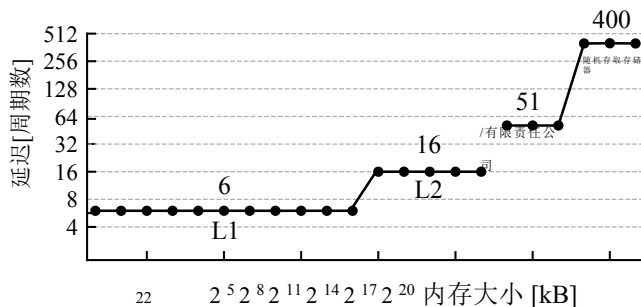


图7. Zen 4架构下不同大小内存区域的中位内存访问延迟，数据以对数坐标轴呈现。平台区内的延迟读数分别表示L1缓存、L2缓存、LLC缓存及RAM的延迟值。

L2驱逐集。 假设L2索引的设计方式使得2MB大页内的内存均匀分布在所有L2缓存集中。因此，需要在2MB页面边界以下至少11位索引位才能映射到全部2048个L2缓存集。基于先前对Zen 3架构缓存逆向工程的研究[26]，我们进一步假设2MB页面边界以下的部分高位未用于L2索引。鉴于2MB页面中共有21位地址位，其中11位用于索引、6位用于64字节缓存行内的偏移量，我们推测最高4位未被用于缓存索引。这将导致每个L2缓存集包含16个条目，我们认为这足以实现对全部8路缓存的驱逐操作。

我们通过以下方法验证上述假设：首先随机选取一个2MB的大页面，根据[6:16]位的数值将该大页面内所有缓存行对齐地址划分为若干集合；随后通过测量重复访问每个集合内所有条目的平均延迟时间，确认这些生成的集合均具备自驱逐特性；进一步通过测量从每个集合中重复访问其半数地址的延迟时间，确保任意两个生成集合之间不存在相互驱逐现象。基于实验结果，我们得出结论：AMD Zen 4架构中的L2缓存索引机制并未使用[17:20]位。

观察结果 O9: 在 AMD Zen 4 架构的 L2 缓存索引过程中，比特位 [17:20] 未被使用。

AMD Zen 4架构的二级缓存（L2）具备8路替换通道，根据最近最少使用（LRU）替换策略，理论上应至少需要8个地址才能完成有效置换。然而实验表明，要可靠地将目标地址从L2缓存中驱逐出去，实际至少需要12个地址。我们认为这种现象源于该替换策略比 LRU 更为复杂。

LLC驱逐集。 基于AMD Zen 4架构的LLC是L2缓存的非包含式受害者缓存。因此，若要将缓存行插入LLC进行驱逐，这些缓存行必须先从L2中被驱逐出去。根据给定的L2驱逐集，我们可以识别出同一1GB巨大页面内的所有地址。

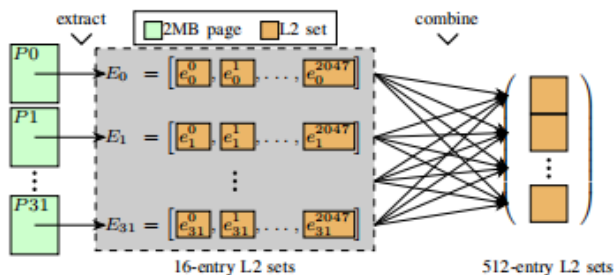


图8. 针对基于虚拟机的攻击者的LLC缓存驱逐集构建策略。首先，我们提取不同2MB页面中包含的L2驱逐集；其次，我们将来自不同2MB页面的L2驱逐集合并。

通过检测这些地址是否被该驱逐集清除，即可确定它们与该驱逐集的对应关系。假设地址在1GB大页内的2048个L2缓存集中均匀分布，我们预计会有8192个地址映射到同一L2缓存集——这一结果已通过实验得到验证。进一步研究证实，其中前512个地址已能为同一L2缓存集内的任意地址提供可靠的LLC驱逐机制。虽然这并非最小规模的驱逐集，但其效率足以满足攻击需求，并显著延长了推测窗口时长，如图6（RAM）所示。

Zen 5: 由于 L1 和 L2 缓存中增加了更多存储单元，缓存驱逐集的大小相应增加。与 Zen 4 相比，Zen 5 减少了 L2 集的数量，使其再次与 Zen 3 相匹配。因此，我们发现 2 MB 分页中的相同位段（即 [16:20]）仍像在 Zen 3 中一样未被使用。最后，与 Zen 4 相比，Zen 5 需要两倍数量的地址，并且必须对每个地址进行两次访问，才能可靠地扩大推测窗口。

7.2. 从虚拟机中退出

我们的LLC驱逐机制使攻击得以成功实施，但该机制采用了精确计数器和1GB页面大小——这两项功能默认情况下对 QEMU 客户端不可用。此外，通过暴力破解新地址集合成员来增大每个L2驱逐集的规模既耗时又易受噪声干扰。首先，我们通过同时测量驱逐集中所有条目的访问时间而非分别测量，从而克服了计时器限制以增强信号强度；其次，我们提出了一种显著更高效的驱逐集生成方法，且无需依赖1GB巨型页面。

我们注意到，Linux默认采用透明的2MB大页机制来支持虚拟机页面，以此提升性能。因此，我们在虚拟机内部构建基于2MB页面的L2缓存集的方法仍然适用。此外，我们认为，在拥有大量此类2MB页面的情况下，可以如图8所示为每个页面提取对应的L2缓存集。虽然这为我们提供了足够的地址空间用于LLC置换策略，但仍需要找到一种高效的方法来整合来自不同页面生成的L2缓存集。

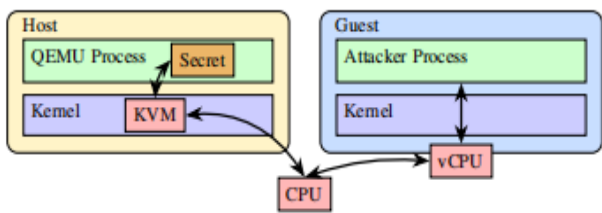


图9. 基于 KVM 的客户机运行于支持硬件虚拟化的CPU上，由 QEMU 管理。

令 e_{ij} 表示源自页面 j 的第 i 个驱逐集，其中 i 由L2索引位[6:16]确定。令 $E_j = \{e_{ij} \mid i \in [0, 2047]\}$ 表示页面 j 的所有驱逐集集合。我们的目标是合并 $E_j(j)$ 中的所有L2集。 $\neq 0$)映射到 E_0 的L2集合中。现在假设对于某个 j 、 a 和 b ，集合 e_{aj} 与集合 e_{bj} 发生碰撞。我们需要高效地确定其将与哪个集合 e_{cj} 发生碰撞。我们假设L2和LLC采用了基于XOR的索引函数，该函数可按以下方式计算出发生碰撞的集合 e_{d0} ：

$$d = a \oplus b \oplus c$$

采用这种方法，可以为每个 $E_j(j)$ 中的单个元素找到匹配的L2集合。 $\neq 0$)足以确定所有其他元素的冲突驱逐集。Zen 3之前的方案[26]需要通过逐一采样地址并测试每个地址是否被驱逐来增大初始L2集的规模。我们的新方法仅需对该过程进行一次操作，利用2MB页面内的整个第一L2集，从而无需进一步测试即可为同一页面中的所有其他32752个地址分配各自的驱逐集。此外，该方法可靠性极高：在100次运行中首次尝试即实现LLC驱逐集创建的100%成功率，而先前研究[26]在使用1GB页面时对Zen 3的报告成功率约为60%。

观察结果O10： 基于 XOR 的缓存索引功能能够高效地将来自不同2MB页面的L2数据集合并为更大的L2数据集。

随着投机窗口的成功延长，我们现在着手构建 VMS CAPE。

8. VMSCAPE

我们在第5节介绍的跨域实验表明，所有AMD Zen CPU和Intel Coffee Lake CPU均易受vBTI_{GU→HU}攻击模式的影响。本节我们将介绍 VMS CAPE——一种作为恶意客户机泄露虚拟机监控程序密钥的端到端攻击工具。图9展示了相关组件架构。我们的攻击目标是 QEMU（Linux生态系统中工业领域常用的虚拟机监控程序_{用户}[15]）。然而，威胁模型（参见第3节）或攻击设计中的任何假设，均无法从根本上阻止使用稍作修改的技术对其他虚拟机监控程序_{用户}实施攻击。我们已在搭载Linux系统的AMD Zen 5服务器CPU上执行了该攻击。

该软件包含内核版本6.8.0-85Generic及Ubuntu 25.10软件包，内置最新 QEMU v10.0.2。我们进一步验证了 VMS CAPE能够在AMD Zen 4系统上成功泄露信息。

在第7节中，我们解决了该攻击所引发的推测窗口问题。

然而，要实现端到端的漏洞利用，还需解决另外两个挑战：

- 首先，我们需要确定一个合适的漏洞利用链，该链由侧信道、推测装置和披露装置组成（详见第8.1节）。
- 其次，我们需要对 QEMU 的地址空间布局随机化（ASLR）进行去随机化处理，以确定受害分支地址及重载缓冲区地址（详见第8.2节）。

最后，我们将提出的解决方案与推测窗口延长技术相结合，构建了一种针对 QEMU 的任意内存泄漏攻击方法（详见第8.3节）。

8.1. 漏洞利用链

我们的目标是在 QEMU 未修改的代码中发现一条漏洞利用链。要构建这样的链，我们需要在 QEMU 二进制文件中找到合适的侧信道、推测装置和披露装置。

侧信道。 我们首先定义用于泄露主机机密的侧信道。通过研究，我们发现 QEMU 将所有客户机内存映射到其虚拟地址空间；主机与客户机之间的共享内存使得F LUSH+R ELOAD成为一种侧信道[20]。

观察结果 O11。 QEMU 将所有客户机内存映射到 Hypervisor_{用户} 虚拟地址空间，从而实现 F LUSH+R ELOAD。

F LUSH+R ELOAD通过检测近期被访问过的内存地址（与受害者共享的）来泄露机密信息。作为侧信道使用的共享内存被称为**重载缓冲区**。

推测装置。 我们的攻击链需要触发推测操作，而vBTI原语通过间接调用或间接跳转来实现这一点。我们在能够对寄存器进行一定程度控制的位置寻找此类推测装置，从而影响被推测目标的执行过程。通过使用F LUSH+R ELOAD，我们旨在控制两个寄存器：一个存储秘密指针，另一个存储重载缓冲区地址。此外，所需的推测装置可以被重复、可靠且高频地触发，从而实现高带宽泄漏。

在管理基于 KVM 的虚拟机时，QEMU 主要负责I/O操作和设备仿真。因此，QEMU 的实际代码量相对较小。我们仅将搜索范围限定在通用I/O处理代码上，避免针对特定设备仿真进行分析，从而不对威胁模型提出额外假设。

如图所示，我们在 QEMU 的内存映射I/O（MMIO）写入处理中找到了合适的受害者分支。

```

1 static MemTxResult memory_region_write_accessor(
2     MemoryRegion *mr,
3     hwaddr addr,
4     uint64_t *value,
5     unsigned size,
6     signed shift,
7     uint64_t mask,
8     MemTxAttrs attrs) {
9     uint64_t tmp = memory_region_shift_write_access(
10         value, shift, mask);
11     // [...]
12     mr->ops->write(mr->opaque, addr, tmp, size);
13     return MEMTX_OK;
14 }

```

Listing 1. Extract from QEMU's system/memory.c source file, showing the victim branch on line 12, with the attacker-controlled value in tmp.

如清单1所示，攻击者能够控制tmp中的值，该值对应于MMIO 操作中写入的数据。虽然攻击者在函数调用时仅直接控制单个64位寄存器（RDX），但动态分析表明，第二个寄存器（R12）中也保留了先前使用的相同数值。此外，我们可以通过共享内存传递额外值，但这需要在披露机制中执行一次额外的加载操作。

观测结果O12： MMIO 实现了客户机与虚拟机监控程序之间可重复、可靠且高频次的寄存器控制交互。

需要注意的是，虽然通用 MMIO 支持8字节写入操作，但被写入的设备也必须支持这种粒度的写入。在我们的主要攻击方案中，我们采用了 QEMU 虚拟机默认提供的hpet高精度事件计时器。

信息泄露装置。 为了从攻击对象处获取机密信息，我们需要一种能够利用 F LUSH+R ELOAD 的信息泄露装置。我们基于 Wikner 和 Razavi [2] 提出的信息泄露装置扫描器进行改进：由于原始扫描器无法在 QEMU 二进制文件中检测到所需装置，我们通过优化针对使用不同污染状态基址寄存器和索引寄存器的内存指令的污染逻辑来扩展该扫描器；同时新增了链式装置搜索功能——将两个独立装置组合起来，其中第一个装置以间接调用结尾，可通过训练使其错误预测第二个装置的行为。这些改进使我们能够成功定位目标信息泄露装置。清单2展示了本攻击中使用的最终装置链。

该装置不会对密钥应用任何乘数，这可能会对 F LUSH +R ELOAD 造成问题。然而，先前的研究表明，我们可以通过采用调整重载缓冲区基址的技术，并检查密钥是否最终位于不同的缓存行中，来克服这一限制 [2]。

分支冲突。 当到达 QEMU 中的目标分支时，控制流刚刚经过一个 VMEXIT 指令后执行了KVM Exit操作。因此，分支历史记录会受到客户机、KVM 和 QEMU 中各分支的影响。

```

1 // first part of the gadget chain
2 0xac334e mov rdi, qword ptr [r12 + 0x2b58]
3 0xac3356 call qword ptr [r12 + 0x2b50]
4
5 // second part of the gadget chain
6 0x8260b5 add cl, byte ptr [rdi]
7 0x8260b7 test dword ptr [rdi + rcx], esp

```

Listing 2. Disclosure gadget in two parts. Line 2 loads the secret pointer, line 6 retrieves one byte of the secret and line 7 accesses the reload buffer.

具体取决于历史缓冲区的大小。由于AMD基于历史记录的分叉预测优先于静态 BTB 预测，因此需要覆盖或作废针对注入目标的历史预测结果才能获得有效观测数据。在训练过程中精确复现历史记录虽可行，但难度较大。根据AMD官方文档[18]所述：[28]仅对那些接触过多个攻击目标的间接分支才会考虑其历史记录。通过在训练前使用 IBPB 清空 BPU，恶意攻击者可确保受害分支仅观察到训练目标。因此，无需参考最后两个相对容易复制的分支之外的历史数据，即可错误预测该分支。尽管允许访问 IBPB 的攻击者看似是一个强有力的假设，但实际上这正是预期中的情况——因为攻击者本能够实现自行实现域隔离。

观察结果O13： 客户端软件可利用对缓解控制措施的访问权限，影响主机软件可使用的分支预测条目。

然而，AMD Zen 4上的Zen 5及后续微代码更新修改了 IBPB，使其同时使 BTB 中的直接分支类型信息失效，从而有助于缓解推测性返回栈溢出（SRSO）[6]。[29]这无意中缩小了我们攻击的推测窗口大小——推测是因为第一个受害者指针加载与受害者分支之间的直接分支会减慢推测执行速度。因此，我们必须在不使用 IBPB 的情况下，让 BTB 忘记受害者分支实际上是一个具有多个目标的间接分支。对于AMD Zen 4和Zen 5架构，我们可以通过以下方式实现：将受害者 BTB 条目替换为虚假的直接分支预测。(1)用直接跳转指令替换训练分支；(2)执行训练流程；(3)将原始训练分支写回内存。此方法符合AMD针对Retbleed漏洞制定的Jmp2Ret缓解方案[30]。

如今，我们能够从 QEMU 中可靠且重复地劫持受害分支，将临时执行流程重定向至我们的披露机制。

8.2. 打破 ASLR

Linux默认使用 ASLR 机制对用户空间代码、堆和栈的虚拟地址进行内存随机化。要实施成功的攻击，我们需要确定推测装置（speculationgadget）和泄露装置（disclosuregadget）的具体位置。

该虚拟地址是 QEMU 映射的虚拟机内存地址，其中存储着我们的重载缓冲区。

部分代码 ASLR 去随机化。 Linux 将 ASLR 实现为相对于程序非 ASLR 基础地址的随机偏移量。最大偏移量是一个可配置参数，其默认值由 Linux 动态确定。目标系统采用默认的最大偏移量为 2^{32} 。

与先前的研究类似，我们的目标是通过检查受害分支的所有可能位置[8]来确定推测装置的地址。[31]他们在攻击者情境下的某个假设受害者位置 V_i 处训练分支 B_i ，触发受害者指令的执行，并测量对 B_i 的预测是否被清除。该方法在我们的案例中不适用，主要基于两个原因：首先，与英特尔处理器不同，AMD Zen 4 和 Zen 5 架构上的 BTB 并非每次预测错误都会覆盖目标地址，而是将新目标地址插入 ITA 中，同时保留 BTB 中的原有目标地址；其次，我们需要分别检查多达约 2^{32} 个位置，这会耗费极长时间。

我们通过反转攻击者与受害者的角色（如先前研究[7]所提出）来优化上述方法。给定假设的受害分支位置 V_i ，我们可以计算出对应的靶地址 T_i 。因此，我们首先触发受害者训练分支预测器，然后在攻击者域的 V_i 位置对某个非信号目标执行匹配分支。通过在攻击者域中假设的受害目标位置 T_i 额外部署一个信号装置，即可检测碰撞事件。采用新方法时，仅需一次受害者分支执行和一次信号相关的重载缓冲区重载操作，即可检查多个潜在受害者位置（例如 1024 个位置）。一旦观测到信号，我们就能确定哪个位置是正确的。

结果。 虽然该方法能在 Zen 4 架构上确定单个受害者的位置——直接破解代码 ASLR，但在 Zen 5 架构上我们获得了多个候选目标。由于所有候选目标共享相同的低 24 位数据，我们认为这是由别名效应以及 BTB 仅提供部分目标所致（与 Intel 方案类似[8]）。[17]虽然这种方法能揭示受害者分支地址的低 24 位信息，但仅对代码 ASLR 进行了部分去随机化处理（该代码负责生成页面偏移位上方的 32 位随机值）。我们发现：当仅使用匹配的低 24 位数据训练预测器时，其输出结果同样仅包含目标地址的低 24 位信息。因此，由于代码 ASLR 仅经过部分去随机化处理，我们只能推测目标地址与受害者分支地址之间的偏移量在 24 位以内。然而，我们所采用的披露装置链位于距离受害者分支地址更远的位置（超过 2^{24} 倍），这要求我们必须完全破解代码 ASLR 才能获取完整信息。

定位重载缓冲区。 在完全实现代码 ASLR 之前，我们需要确定 QEMU 为攻击者虚拟机使用的内存所映射的虚拟地址，该地址用于存储重载缓冲区。我们还需要通过 F LUSH+R ELOAD 最终向重载缓冲区泄露任意数据。与先前研究[2]类似，我们采用暴力搜索法确定重载缓冲区的位置，并进行了部分去随机化处理。

基于先前的研究结果，我们可以推测目标地址位于受害者分支指令产生的 24 位偏移范围内。在该区域内，我们发现了一个能够加载攻击者指定内存地址的机制。值得注意的是，正如先前观察到的那样，Linux 与 QEMU 系统会将虚拟机页面映射为 2MB 的大页面。因此，通过将重载缓冲区映射为虚拟机内的 2MB 大页面，我们可以显著减少潜在的重载缓冲区位置数量。此外，由于搜索重载缓冲区仅需在推测窗口内执行一次加载操作，这意味着我们需要相应延长此步骤的推测窗口长度。

结果。 经过 100 次重复实验，我们能够以 100% 的成功率实现重载缓冲区位置的去随机化处理，在 Zen 5 和 Zen 4 处理器上的耗时中位数分别为 46 秒和 83 秒。

完全破解代码 ASLR。 在定位到重载缓冲区后，我们现在可以彻底消除代码 ASLR 的随机性并确定推测指令 `getdget` 的位置。这一过程同时揭示了泄露装置的位置——因为 ASLR 仅对可执行文件的基地址进行随机化处理，而保留了偏移量信息。对于 Zen 4 架构而言，此额外步骤并非必需，因为其 BTB 架构未表现出别名现象，因此可直接获取受害者内存位置。

在受害者分支处，寄存器 `RAX` 包含指向某个代码位置的指针。掌握该值可让我们计算出完整的受害者分支地址。我们可以通过位于受害者分支 24 位偏移量内的一个额外机制来泄露此指针：该机制将攻击者控制的寄存器 `RDX` 与 `RAX` 相加，并加载所得地址。通过暴力尝试 `RDX` 的不同取值，我们可泄露 `RAX` 的地址，从而在重载缓冲区中记录命中结果。

结果。 在 Zen 5 架构上，完全破解 ASLR 码的中位耗时为 12 秒，准确率为 75%（未计入定位重载缓冲区所需时间）；而在 Zen 4 架构上，由于第一步已能确定受害者分支的位置，破解 ASLR 码耗时 238 秒，准确率为 70%。时间差异源于：在 Zen 5 架构的代码 ASLR 破解第一步中，只需从 2^{12} 个候选地址中找到一个有效目标；而 Zen 4 架构则需检查全部 2^{32} 个可能地址才能定位正确位置。Zen 4 架构第一步所需的搜索空间更大，导致其耗时显著长于 Zen 5 架构所需的第二步——后者通过泄露 ASLR 熵的剩余 20 位信息来实现破解。

8.3. 任意内存泄漏

我们现在已具备构建任意内存泄漏攻击所需的所有组件：漏洞利用链、ASLR 破解原语，以及一种能够提供足够大推测窗口的可靠方法。

我们的 VMS CAPE 攻击流程如下：首先，我们利用 ASLR 破坏技术定位目标分支；确定相关 gadget 后，即可获取 QEMU 重载缓冲区的虚拟地址。接着构建 LLC 驱逐集，并通过测试哪个驱逐集能使依赖性加载操作产生缓存跟踪痕迹来确定正确集合；最后结合长推测窗口机制，利用这些 gadget 泄露 QEMU 的内部数据结构。

找到秘密数据的地址，然后泄露该秘密数据。

结果。对Zen 5芯片实施完整的端到端攻击所需中位时间为102秒，可泄露一个4 KiB的QEMU密钥对象（以磁盘加密/解密密钥为例）。在总攻击时间内，需要58秒即可定位受害分支和重载缓冲区。获取这些信息后，我们的漏洞利用工具能以154字节/秒的速度泄露任意QEMU内存，字节精度达99.85%。该攻击的整体成功率为68%。

9. 讨论

我们讨论了针对VMS CAPE攻击的NUMA相关考量（第9.1节）、我们在首次负责任披露后发现的其他vBTI变体（第9.2节），以及针对vBTI原语的缓解策略（第9.3节）。

9.1. NUMA考虑事项

我们发现，在采用NUMA技术的Zen 5系统中，VMS CAPE无法访问某些内存页面。经深入排查，发现Linux的NUMA均衡机制[32]会定期将页面标记为“不存在”，以追踪这些页面是否被同一NUMA节点内的CPU核心频繁访问。被标记为“不存在”的页面无法通过推测执行机制进行访问——这实际上阻止了VMS CAPE对虚拟机监控程序不常访问的内存页面进行操作。然而，云虚拟化平台和提供商通常会在同一NUMA节点上为虚拟机分配物理内存以提升性能[33]。[34]，[35]在这样的实际应用场景中，Linux内核建议关闭NUMA平衡功能以减少不必要的跟踪开销[32]。

9.2. BHI 变体

英特尔对经典Spectre-BTI [1]的应对方案是eIBRS——一种能有效隔离BTB中用户与管理权限条目的硬件缓解措施。然而，分支历史注入（BHI）[3]通过利用用户域与管理权限域之间缺乏BHB隔离的漏洞，重新激活了跨权限的Spectre攻击。如第5节所示，eIBRS在虚拟化环境中成功实现了BTB隔离。但与BHI类似，eIBRS本身并不能完全阻止客户机对分支历史进行（部分）控制。虽然针对HS的攻击对此问题有所缓解[5]，[36]我们认为，该漏洞（HU）目前尚无缓解方案。尽管我们未对此进行核实，但英特尔已确认情况确实如此，这表明近期生产的英特尔CPU可能易受基于虚拟化的Spectre-BHI（vBHI）攻击原理的影响。

9.3. 缓解；减轻

如第5节所述，我们的逆向工程揭示了近期多种微架构中存在多个vBTI基本单元。本节将探讨相应的缓解策略。

需防止此类原始漏洞，首先是从IBPB-on-VMEXIT开始——该漏洞正是已部署的VMS CAPE补丁的基础。

IBPB-on-VMEXIT。该缓解措施通过在每个VMEXIT操作时使用IBPB刷新BPU状态，从而在TM_{G→H}模型中保护主机。因此，它确保了每次从客户机到主机的转换过程中实现客户机与主机之间的BTB隔离。然而，基于IBPB的缓解措施通常会带来显著的性能开销[2]。[6]当应用于VMEXIT等快速路径时，这一问题尤为突出。

为了量化这一开销，我们使用UnixBench测试套件[37]对缓解措施进行了基准测试，具体方法遵循了先前研究[2]的方法论。[6]，[8]我们在一个配备4GB内存和两个核心的虚拟机环境中，以多线程模式运行工作负载。在Zen 5处理器上，性能开销（基于五次重复测试）的几何平均值为57%。

Linux内核开发者基于IBPB-on-VMEXIT方案制定了缓解措施，并通过仅在伴随客户机执行的KVM_EXIT操作时触发IBPB来对其进行优化。由于并非所有VMEXIT都会导致KVMEXIT及后续用户空间执行，此改进将IBPB移出快速路径，同时保留其安全保证。他们将该缓解措施称为“退出用户空间前触发IBPB”。该补丁适用于所有受影响系统，包括AMD Zen 5架构处理器以及Lunar Lake和Granite Rapids等最新款Intel CPU。

除了验证补丁的有效性外，我们还在相同测试环境中对其性能开销进行了基准测试。UnixBench结果显示，在Zen 4架构上仅产生1%的微小开销。然而，作为面向计算性能的基准测试工具，UnixBench导致用户空间退出次数相对较少。为了评估那些会频繁触发用户空间退出的工作负载，我们使用fio[38]生成磁盘I/O数据——通过在虚拟化磁盘上随机读写10GB数据并重复执行10次操作。在Zen 4架构下，优化后的缓解方案开销达到51%，接近IBPB-on-KVM_EXIT方案57%的开销水平。当使用由主机内核（vhost）或直通设备管理的设备时，用户空间退出会被降至最低。这些结果表明：该缓解措施对性能的影响既取决于工作负载类型，也受QEMU配置影响；但在大多数实际应用场景中，其影响预计仍处于次要地位。

Retpoline。Retpoline是一种经典的基于软件的SpectreBTI缓解方案，它通过使用不同的预测器[39]预测出的其他间接分支（ret），来替换某些间接分支（jmp和call）。在未受到早期针对返回点攻击影响的系统上，使用retpoline编译Hypervisor_{用户}程序可使其免受G→H|Hypervisor_{用户}场景下的攻击者侵害[6]。虽然该方案无法保护涉及G→H|Host Process的其他主机用户进程，但其产生的性能开销低于IBPB-on-VMEXIT。使用UnixBench工作负载对retpoline的性能评估显示开销为3.9%。根据威胁模型的不同，将retpoline与使用PR_SET_SPECULATION_CTRL prctl实现的选择性主机进程隔离相结合，可作为高开销IBPB-on-VMEXIT的替代策略。我们的研究表明

目前流行的开源Hypervisor用户均未部署retpoline[13]。[40], [41], [42]该缓解措施仅适用于不受Phantom推测影响的CPU[43], 例如Zen 5。

保护 SEV -SNP客户机。虽然受vBTI原语影响的英特尔系统不支持 TDX, 但Zen 3至Zen 5架构上的 SVM 为启用 SEV -SNP的客户机提供了多种可选保护机制[18]。[28](1) *分支目标缓冲区隔离* 技术可防止宿主域外部的执行操作影响宿主执行环境内的分支决策。(2) *入口处的间接分支预测屏障* 机制迫使CPU在每次 VMRUN 指令执行时都必须发出 IBPB 指令。(3) *SMT 保护机制* 会强制使兄弟 SMT 线程处于空闲状态, 而宿主操作则在另一个兄弟线程上执行。我们在第5节的实验表明, Zen 5架构默认已为 SEV -SNP宿主提供足够的隔离能力。因此, 此类防护措施在早期架构的CPU上同样适用。

保护 SMT。STIBP 是一种硬件缓解措施, 可隔离 SMT 核心的 BPU。我们发现大多数系统默认启用 STIBP 功能, 从而防范vBTI- SMT 原语的影响。然而, 在某些CPU (如Coffee Lake) 中, STIBP 并非始终处于开启状态。对于不支持 STIBP 的系统 (例如Zen 1架构), 唯一可行方案是完全禁用 SMT, 但这会导致性能显著下降[2]。[44]。

基于硬件的隔离。最终, vBTI原语的根本原因在于主机域与客户域之间缺乏 BPU 状态隔离。Zen 5处理器引入了特权域标记机制以区分用户域和管理员域; 采用类似的标记机制来区分主机域与客户域可有效防止vBTI原语的发生, 这一方案应纳入Zen微架构的后续版本更新中。

10. 相关工作

关于微架构安全性的研究已有数十年的深厚积淀, 其研究范围始于时序侧信道分析, 尤其聚焦于缓存领域[45]。[46]。Aciic ,mez等人[47], [48] 发表了类似的研究, 但其研究对象为早期分支预测因子。Evtyushkin等人[31] 后续研究表明, 现代BTB技术同样可用于破解 ASLR。Ge等人[49] 本文提出了一种针对此类 (及其他多种) 可通过共享资源与上下文利用的时序侧信道的分类方法。

Spectre。Kocher等人[1] 首次提出了由分支预测错误引发的瞬态执行攻击。随后又出现了大量其他类型的瞬态执行攻击[50]、[51], [52],[53], [54], [55], [56], [57] Canella等人[58] 对这类攻击进行了评估与分类。根据受害者的防护领域, 我们将先前提出的推测性执行攻击分为三类: (1)虚拟机监控程序攻击; (2)管理程序攻击; (3)用户空间攻击。

(1) 虚拟机监控程序攻击。关于 Spectre 的原始研究[1], 以及后续的研究[5], [8]我们针对虚拟机监控程序软件中经过修改的监督组件进行了概念验证攻击。谷歌研究人员基于Inception[6], 部分公开了针对 KVM 的攻击方案[59]。我们的研究工作.....

相比之下, 这是首个真正的端到端攻击案例: 恶意 KVM 客体专门针对虚拟机监控程序的用户空间组件发起攻击。

(2) 监督者攻击。监督者模式自早期起便是 BTI 漏洞利用的热门目标[1]。尽管CPU厂商和操作系统开发者争相开发防护措施, 研究人员仍不断发现新防御机制中的漏洞。Wikner与Razavi通过推翻预测器训练的基本假设, 在Retbleed[2]、Phantom[43]及Inception[6]中规避了retpoline线机制与硬件级防护手段。BHI 及其后续研究反复证明: 即便存在多种防范措施, 同时模式训练仍是可行的[3]。[4], [5]这打破了基于交叉权限 BTI 缓解措施所建立的假设。

(3) 用户空间攻击。已有多种针对用户空间的攻击方法被提出, 尤其是针对浏览器沙箱技术的攻击。Ragab等人[60] 广义机器清除被确认为推测性执行的来源, 并发现了他们在浏览器中利用的新型机器清除诱因。Ret2spec [61] 和 Spectre Returns [21] 均通过探索返回栈缓冲区 (RSB) 预测机制, 在多种场景下攻击用户空间受害者。Spring [62] 随后证明, 尽管存在各种系统级和浏览器级防护措施, RSB 预测仍可被利用。更近期的攻击还针对其他预测器, 同样实现跨沙箱边界的信息泄露[63], [64]。Wikner与Razavi [7] 首次完整展示了端到端的跨进程攻击方案。与以往仅针对用户空间的攻击不同, 我们识别并利用了不仅能跨上下文操作, 还能同时跨越权限与虚拟化边界的原语。

11. 结论

我们提出了 VMS CAPE——这是首个在 QEMU 平台上由恶意 KVM 客户机发起的实际SpectreBTI攻击, 可在AMD Zen 5处理器上以154字节/秒的速度泄漏内存。与以往由恶意客户机发起的Spectre- BTI 攻击不同, VMS CAPE无需对任何虚拟机监控程序代码进行修改。VM S CAPE通过一种新颖的vBTI原语实现这一目标: 该原语是我们通过对不同AMD和Intel CPU上、不同权限级别下客户机与主机之间分支预测器状态的隔离间隙进行系统性探索而发现的。缓解 VMS CAPE 攻击需要在进入用户空间虚拟机监控程序时执行 VMEXIT 后的 IBPB 操作, 这会带来轻微的性能开销。

致谢

我们谨向匿名审稿人及我们的指导老师致以诚挚谢意, 感谢他们提供的宝贵反馈。同时, 我们也衷心感谢英特尔 PSIRT、AMD PSIRT 和Linux安全团队在该项目中的协调与合作。此外, 特别感谢谷歌的Alexandra Sandulescu对我们缓解方案分析提出的宝贵意见。本研究由瑞士教育、研究与创新部根据合同编号MB22.00057 (ERC-StG承诺) 资助完成。

References

- [1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre Attacks: Exploiting Speculative Execution," in *2019 IEEE Symposium on Security and Privacy (SP)*, May 2019, pp. 1–19.
- [2] J. Wikner and K. Razavi, "RETBLEED: Arbitrary Speculative Code Execution with Return Instructions," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3825–3842.
- [3] E. Barberis, P. Frigo, M. Muench, H. Bos, and C. Giuffrida, "Branch History Injection: On the Effectiveness of Hardware Mitigations Against Cross-Privilege Spectre-v2 Attacks," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 971–988.
- [4] S. Wiebing, A. d. F. Tron, H. Bos, and C. Giuffrida, "InSpectre Gadget: Inspecting the Residual Attack Surface of Cross-privilege Spectre v2," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 577–594.
- [5] S. Wiebing and C. Giuffrida, "Training Solo: On the Limitations of Domain Isolation Against Spectre-v2 Attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, May 2025, pp. 3599–3616.
- [6] D. Trujillo, J. Wikner, and K. Razavi, "Inception: Exposing New Attack Surfaces with Training in Transient Execution," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 7303–7320.
- [7] J. Wikner and K. Razavi, "Breaking the Barrier: Post-Barrier Spectre Attacks," in *S&P*, May 2025.
- [8] S. Rügge, J. Wikner, and K. Razavi, "Branch Privilege Injection: Compromising Spectre v2 Hardware Mitigations by Exploiting Branch Predictor Race Conditions," in *USENIX Security*, Aug. 2025, intel Bounty Reward, BlackHat USA presentation. [Online]. Available: [Paper=https://comsec.ethz.ch/wp-content/files/bprc_sec25.pdf](https://comsec.ethz.ch/wp-content/files/bprc_sec25.pdf) [URL=https://comsec.ethz.ch/bprc](https://comsec.ethz.ch/bprc)
- [9] M.-M. Bazm, M. Lacoste, M. Südholt, and J.-M. Menaud, "Isolation in cloud computing infrastructures: New security challenges," *Annals of Telecommunications*, vol. 74, no. 3, pp. 197–209, Apr. 2019.
- [10] Advanced Micro Devices, Inc., "AMD64 APM, Volume 2: System Programming," Tech. Rep. 24593, Mar. 2024.
- [11] "Intel Trust Domain Extension," Tech. Rep.
- [12] "Kernel virtual machine," https://linux-kvm.org/page/Main_Page.
- [13] "QEMU: A generic and open source machine emulator and virtualizer," <https://www.qemu.org/>.
- [14] "Intel64 SDM, Volume 3 (3A, 3B, 3C & 3D): System Programming Guide," Tech. Rep.
- [15] "Proxmox: Simplify your data center," <https://proxmox.com/en/>.
- [16] H. Yavarzadeh, A. Agarwal, M. Christman, C. Garman, D. Genkin, A. Kwong, D. Moghimi, D. Stefan, K. Taram, and D. Tullsen, "Pathfinder: High-Resolution Control-Flow Attacks Exploiting the Conditional Branch Predictor," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. La Jolla CA USA: ACM, Apr. 2024, pp. 770–784.
- [17] L. Li, H. Yavarzadeh, and D. Tullsen, "Indirector: High-Precision Branch Target Injection Attacks Exploiting the Indirect Branch Predictor," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2137–2154.
- [18] Advanced Micro Devices, Inc., "Software Optimization Guide for the AMD Zen5 Microarchitecture," Tech. Rep. 58455, Aug. 2024.
- [19] Y. Zhang and R. Sion, "Speculative Execution Attacks and Cloud Security," in *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*, ser. CCSW'19. New York, NY, USA: Association for Computing Machinery, Nov. 2019, p. 201.
- [20] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 719–732.
- [21] E. M. Koruyeh, K. N. Khasawneh, C. Song, and N. Abu-Ghazaleh, "Spectre Returns! Speculation Attacks using the Return Stack Buffer," in *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.
- [22] "Indirect Branch Restricted Speculation," Tech. Rep.
- [23] Advanced Micro Devices, Inc., "Amd64 Technology Indirect Branch Control Extension," Tech. Rep., Jul. 2018.
- [24] "Single Thread Indirect Branch Predictors," Tech. Rep.
- [25] "Providing secret data to QEMU," <https://www.qemu.org/docs/master/system/secrets.html>.
- [26] H. Wang, M. Tang, Q. Wang, K. Xu, and Y. Zhang, "ZenLeak: Practical Last-Level Cache Side-Channel Attacks on AMD Zen Processors," 2025.
- [27] Advanced Micro Devices, Inc., "AMD64 APM, Volume 3: General-Purpose and System Instructions," Tech. Rep. 24594, Mar. 2024.
- [28] —, "Software Optimization Guide for the AMD Zen4 Microarchitecture," Tech. Rep. 57647, Jan. 2023.
- [29] "Speculative Return Stack Overflow (SRSO) — The Linux Kernel documentation," <https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/srso.html>.
- [30] Advanced Micro Devices, Inc., "Technical Guidance for Mitigating Branch Type Confusion," Tech. Rep.
- [31] D. Evtyushkin, D. Ponomarev, and N. Abu-Ghazaleh, "Jump over ASLR: Attacking branch predictors to bypass ASLR," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2016, pp. 1–13.
- [32] Documentation for /proc/sys/kernel. The Linux Kernel documentation. [Online]. Available: <https://docs.kernel.org/admin-guide/sysctl/kernel.html#numa-balancing>
- [33] Configure NUMA-aware VMs — Google Distributed Cloud (software only) for bare metal. Google Cloud. [Online]. Available: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/vm-runtime/numa>
- [34] How ESXi NUMA Scheduling Works. [Online]. Available: <https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0/vsphere-resource-management-8-0/using-numa-systems-with-esxi/how-esxi-numa-scheduling-works.html>
- [35] NUMA - Proxmox VE. [Online]. Available: <https://pve.proxmox.com/wiki/NUMA>
- [36] "Branch History Injection and Intra-mode Branch Target Injection," <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/branch-history-injection.html>.
- [37] "UnixBench test suite," <https://github.com/kdlucas/byte-unixbench>.
- [38] "Flexible I/O Tester," <https://github.com/kdlucas/byte-unixbench>.
- [39] "Retpoline: A software construct for preventing branch-target-injection," <https://support.google.com/faqs/answer/7625886>.
- [40] "VirtualBox: Powerful open source virtualization For personal and enterprise use," <https://www.virtualbox.org/>.
- [41] "Firecracker: Secure and fast microVMs for serverless computing."
- [42] "Crosvm: The ChromeOS Virtual Machine Monitor."
- [43] J. Wikner, D. Trujillo, and K. Razavi, "Phantom: Exploiting Decoder-detectable Mispredictions," in *56th Annual IEEE/ACM International Symposium on Microarchitecture*. Toronto ON Canada: ACM, Oct. 2023, pp. 49–61.
- [44] P. Michaud and A. Seznec, "A Comprehensive Study of Dynamic Global History Branch Prediction," Report, INRIA, 2001.

- [45] W.-M. Hu, “Lattice scheduling and covert channels,” in *Proceedings 1992 IEEE Computer Society Symposium on Research in Security and Privacy*, 1992, pp. 52–61.
- [46] P. C. Kocher, “Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems,” in *Advances in Cryptology — CRYPTO ’96*, N. Koblitz, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 104–113.
- [47] O. Aciğmez, Ç. K. Koç, and J.-P. Seifert, “Predicting secret keys via branch prediction,” in *Topics in Cryptology – CT-RSA 2007*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 225–242.
- [48] O. Aciğmez, c. K. Koç, and J.-P. Seifert, “On the power of simple branch prediction analysis,” in *Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security*, ser. ASIACCS ’07. New York, NY, USA: Association for Computing Machinery, 2007, p. 312–320. [Online]. Available: <https://doi.org/10.1145/1229285.1266999>
- [49] Q. Ge, Y. Yarom, D. Cock, and G. Heiser, “A survey of microarchitectural timing attacks and countermeasures on contemporary hardware,” 2018.
- [50] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, “Meltdown: Reading Kernel Memory from User Space,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 973–990.
- [51] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, “Fore-shadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, p. 991.
- [52] V. Kiriakos and C. Waldspurger, “Speculative buffer overflows: Attacks and defenses,” 2018. [Online]. Available: <https://arxiv.org/abs/1807.03757>
- [53] J. Stecklina and T. Prescher, “Lazyfp: Leaking fpu register state using microarchitectural side-channels,” 2018. [Online]. Available: <https://arxiv.org/abs/1806.07480>
- [54] D. Moghimi, “Downfall: Exploiting speculative data gathering,” in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 7179–7193. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/moghimi>
- [55] S. van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, “RIDL: Rogue in-flight data load,” in *S&P*, May 2019.
- [56] C. Canella, D. Genkin, L. Giner, D. Gruss, M. Lipp, M. Minkin, D. Moghimi *et al.*, “Fallout: Leaking data on meltdown-resistant cpus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2019.
- [57] M. Schwarz, M. Lipp, D. Moghimi, J. Van Bulck, J. Stecklina, T. Prescher, and D. Gruss, “ZombieLoad: Cross-privilege-boundary data sampling,” in *CCS*, 2019.
- [58] C. Canella, J. V. Bulck, M. Schwarz, M. Lipp, B. von Berg, P. Ortner, F. Piessens, D. Evtushkin, and D. Gruss, “A systematic evaluation of transient execution attacks and defenses,” in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 249–266. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/canella>
- [59] “Google CPU security research,” <https://github.com/google/security-research/tree/master/pocs/cpus>.
- [60] H. Ragab, E. Barberis, H. Bos, and C. Giuffrida, “Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks,” in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1451–1468. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/ragab>
- [61] G. Maisuradze and C. Rossow, “ret2spec: Speculative execution using return stack buffers,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2109–2122. [Online]. Available: <https://doi.org/10.1145/3243734.3243761>
- [62] J. Wikner, C. Giuffrida, H. Bos, and K. Razavi, “Spring: Spectre returning in the browser with speculative load queuing and deep stacks,” in *WOOT*, 2022.
- [63] J. Kim, D. Genkin, and Y. Yarom, “Slap: Data speculation attacks via load address prediction on apple silicon,” in *S&P*, 2025.
- [64] J. Kim, J. Chuang, D. Genkin, and Y. Yarom, “Flop: Breaking the apple m3 cpu via false load output predictions,” in *USENIX Security*, 2025.